

CS180 Project 1: Colorizing the Prokudin-Gorskii Collection

Author: Zhang Zhendan

Overview

In this project, I implemented an algorithm to automatically align and colorize the digitized glass plate images from the Prokudin-Gorskii collection. Each image contains three monochrome channels (Blue, Green, Red) stacked vertically. The task is to properly align them to reconstruct a full RGB color image.

Algorithm

Similarity Metrics

To compare how well two channels are aligned, I tested both **Sum of Squared Differences (SSD)** and **Normalized Cross-Correlation (NCC)**.

$$SSD(I_1, I_2) = \sum_{x,y} (I_1(x, y) - I_2(x, y))^2$$
$$NCC(I_1, I_2) = \frac{\sum_{x,y} (I_1(x, y) - \mu_1)(I_2(x, y) - \mu_2)}{\sqrt{\sum_{x,y} (I_1(x, y) - \mu_1)^2} \cdot \sqrt{\sum_{x,y} (I_2(x, y) - \mu_2)^2}}$$

Here μ_1, μ_2 are the mean pixel values of the two images. SSD measures absolute difference (smaller is better), while NCC measures correlation (closer to 1 is better).

Border Cropping

Directly computing similarity over the full images gave poor results, because noisy or damaged edges dominated the metric. To fix this, I cropped ~30% of the borders before scoring.

Pyramid Alignment

Brute-force searching over large displacements is slow for high-resolution `.tif` files. To improve speed, I used a **pyramid approach**:

1. Construct a pyramid by repeatedly downsampling the image by a factor of 2 until the smallest dimension < 64 pixels.
2. Align the channels at the coarsest (smallest) level with a small search window.
3. At each higher resolution, double the displacement estimate and refine it locally within $\pm$ search_range pixels.

This “coarse-to-fine” strategy reduces runtime from minutes to seconds while keeping high accuracy.

Edge-based NCC

For difficult images like **Emir**, intensity differences across channels caused NCC to fail. To address this, I computed similarity on **Sobel edge maps** instead of raw intensity:

$$E(x, y) = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

Then NCC was applied to these edge maps, making alignment robust to brightness differences.

Results

Below are the reconstructed color images and their computed displacements (dx, dy).

Image	Offsets (Green vs Blue)	Offsets (Red vs Blue)
Cathedral	(5, 2)	(12, 3)
Church	(25, 4)	(58, -4)
Emir	(49, 24)	(107, 41)
Harvesters	(60, 18)	(123, 14)
Icon	(41, 18)	(90, 23)
Italil	(38, 22)	(77, 36)
Lastochikino	(-3, -2)	(75, -8)
Lugano	(41, -16)	(93, -29)
Melons	(81, 10)	(178, 14)
Monastery	(-3, 2)	(3, 2)
Self-Portrait	(78, 29)	(175, 37)

Siren	(49, -5)	(96, -24)
Three Generations	(52, 13)	(110, 10)
Tobolsk	(3, 3)	(7, 3)
master-pnp-prok-00000-00087a	(48, 58)	(108, 74)
master-pnp-prok-00100-00178a	(21, 27)	(110, 53)
master-pnp-prok-00200-00220a	(45, -13)	(102, -11)

Sample Results

Cathedral (aligned)

Emir (with Edge-NCC)



Three examples of my own choosing

Woman in traditional dress and jewelry standing on rug in front of yurt



Etruscan vases in the Hermitage in Saint Petersburg



On the island of Capri



Discussion

The main challenges were efficiency on large .tif images and robustness on difficult cases like Emir. Cropping and pyramids solved the speed issue, while Edge-NCC solved the robustness issue. Overall, the algorithm performs well across the dataset, even on tricky images.

[← Back to Portfolio Home](#)

Created for CS180 – Fall 2025